TABLET CLASSIFICATION APP

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Josh Thompson - G00422756

Institution: Atlantic Technological University Galway
Course: B.Eng(H) in Software and Electronic Engineering
Module: Project Engineering
Supervisor: Paul Lennon
Academic Year: 2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University, Galway.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Signed: Josh Thompson

Date: April 2026

Acknowledgements

I would really like to thank my project supervisor, Paul Lennon, for his guidance, technical feedback, and continuous support throughout my completion of this project. His direction was invaluable at every stage, from ideation to problem-solving, to development. Paul helped me overcome the various technical challenges that arose along the way, particularly during the machine learning and backend integration phases.

I would also like to thank the Project Engineering lecturers at Atlantic Technological University, Galway, for running the Project Engineering labs all year long and always being able to answer any questions I had, relating to the project. Their teaching and feedback throughout the project labs, and academic year helped so much to shape the direction of this project and provided the frameworks I used for project management, documentation, and report writing, as well as the technical side of my project.

I would like to acknowledge Fergus Downey, whose Mobile App Development lab work on React Native and Expo Go provided a practical foundation for the development of my application's frontend.

I want to thank Ben Kinsella for running our Project Engineering tutorials all year. I found them very helpful for staying on track with work to be completed each week. It also really helped to hear what everyone else was working on at the same time, so I could discuss different approaches and solutions to problems I was having with other people in my class, thanks to the tutorials.

Finally, I would like to acknowledge my classmates, for helpful discussions around machine learning model training approaches that contributed to a key improvement in my model's performance.

Table of Contents

1 Summary
2 Poster
3 Introduction
4 Research and Background
4.1 Machine Learning and Convolutional Neural Networks
4.2 CNN Layer Types
4.3 Training Deep Neural Networks
4.4 Transfer Learning and ResNet50
4.5 Existing Similar Products
5 Project Plan and Management
5.1 Methodology
5.2 Sprint Overview
5.3 Challenges and Adaptations
6 System Architecture
6.1 Overall Architecture
6.2 Frontend
6.3 Backend
6.4 Database
6.5 Machine Learning Model
7 Implementation
7.1 Backend and API Development
7.2 Authentication and JWT
7.3 Tablet Schedule Endpoints
7.4 Machine Learning Model Development
7.4.1 Dataset Selection
7.4.2 Initial CNN Development
7.4.3 Hyperparameter Tuning
7.4.4 Class Reduction and Visualisation
7.4.5 ResNet50 Transfer Learning
7.4.6 Final Model Configuration
7.5 Frontend Development
7.6 Prediction Endpoint Integration
7.7 Intake Logging
7.8 API Testing
8 Results and Testing
8.1 Machine Learning Results

8.2 API Test Results
8.3 End-to-End Integration Testing
9 Ethics
10 Future Work
11 Conclusion
12 References
Appendix A: Weekly Development Logs Summary
Appendix B: Software and Technologies Used

1 Summary

The Tablet Classification App is a mobile application designed to help people, particularly those with memory difficulties, identify their medication and manage their tablet-taking schedule. The motivation behind the project stems from a personal desire to support people such as my Granny, who struggles to remember which tablets she needs to take and when, as a result of dementia. The application uses a trained machine learning model to classify tablet images captured or uploaded from a mobile device, and cross-references the identified tablet against the user's personalised schedule to inform them whether the medication is currently required.

The system is built across four primary components. The mobile frontend is a React Native application built with the Expo framework and written entirely in JavaScript. The backend is a Python FastAPI application exposing a RESTful API, containerised with Docker and served with Uvicorn. The database is a PostgreSQL relational database storing user accounts, tablet schedules, and intake logs. The machine learning model is a Keras deep learning model based on the ResNet50 architecture, trained in Google Colab using a dataset of 4,000 synthetic tablet images representing four tablet classes.

The machine learning model was developed through an extensive, iterative process spanning approximately ten weeks of the second semester. This involved experimenting with a custom-built CNN trained from scratch, tuning hyperparameters including batch size, epoch count, and data split ratios, and ultimately adopting a transfer learning approach with a pretrained ResNet50 backbone. The final four-class model, classifying Bactidol, Bioflu, DayZinc, and Medicol, achieved strong classification accuracy with an F1 score of 1.00 for all four classes in the final training run, and demonstrated correct end-to-end prediction when tested through the frontend integration.

The backend was developed as a containerised FastAPI service with PostgreSQL as the data store, using SQLAlchemy as the ORM layer. Secure user authentication is implemented using JSON Web Tokens (JWTs) with bcrypt password hashing. The frontend provides screens for user registration, login, schedule creation and viewing, tablet prediction, and intake logging.

This report documents the full project journey from initial concept exploration through research, design, implementation, testing, and reflection.

2 Poster

Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University

Tablet Classification App

Josh Thompson · G00422756
Project Engineering · Final Year Project 2025/26

B.Eng. (Hons) Software and Electronic Engineering
ATU Galway · Supervisor: Paul Lennon

Project Overview

The Tablet Classification App is a mobile app used for identifying tablets. This project uses machine learning to help the user identify a tablet that they do not recognise. I have completed this by training the machine learning model in Google Colab on a dataset of 4000 tablet images and building a mobile app for the user to use this. My aim is to help people with memory problems to take control over this aspect of their lives.

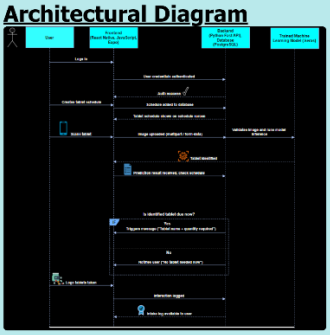
How it works:

1. User signs up/logs in to an account.
2. The user can navigate to the schedules screen where they create a tablet schedule which includes tablet names, time it's due and the quantity required.
3. The tablet prediction screen allows the user to open the phone camera or upload an image of a tablet to the prediction endpoint to identify the image.
4. When the tablet is identified the user is told if it is required at this time, according to their schedule.
5. If the tablet is taken, the user can log this activity with one press of a button to a PostgreSQL database, so the user can track their own intake status.

Machine Learning

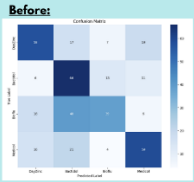
Machine learning model was trained in Google Colab using a dataset which I imported from Kaggle of 4000 tablet images. I have loaded in a pretrained ResNet50 model to achieve high accuracy for classification and detection, rapidly. Using a pretrained model saves significant training time as the model already understands basic visual features like edges and shapes.

Architectural Diagram

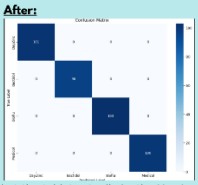


Results

Before:

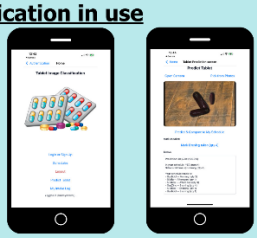


After:



The Y-axis of the confusion matrix shows what the tablet actually is, the X-axis shows what the model has predicted the tablet as. The confusion matrix is plot based on the results of the 400 images used for testing The image on the right is the desirable end goal which I achieved after many weeks of bug-fixing and hyperparameter tuning.

Application in use



The image on the left is the home screen that the user will see once logged in, giving them options to view their tablet schedule, intake log, or to scan a tablet to identify it. The image on the right is the prediction screen, where the user can open the camera or upload an image to have it identified

Software used

- FastAPI: Python web framework used to build APIs in Visual Studio Code.
- React Native: Frontend framework for building user interfaces.
- JavaScript: Frontend programming language.
- Expo: A full-stack React Native framework.
- PostgreSQL: Database.
- Keras: Deep learning API used to build, train and deploy neural networks in Google Colab.

Conclusion

The Tablet Classification App successfully demonstrates that machine learning can be used to help people with memory difficulties manage their medication safely. The core goal of reducing human error in tablet-taking has been achieved, and the project demonstrates a practical, real-world application of machine learning in healthcare.

3 Introduction

The motivation for undertaking this project arose from a very personal place. My Granny has difficulty remembering which tablets she is supposed to take and when. This is not an uncommon problem; many elderly individuals and people with memory-related conditions, such as Alzheimer's disease or mild cognitive impairment, struggle significantly to manage their medication correctly. Incorrect or missed doses can have serious health consequences, and the cognitive burden of managing a complex medication regimen is substantial.

The initial concept for this project was a hardware-based automated tablet dispenser, similar in concept to products such as the Hero Smart Dispenser. After conducting research into existing hardware solutions and consulting with a supervisor, it became clear that hardware projects carry significant time risk; The physical construction work alone could consume the too much development time without leaving enough time for the software aspects, which are at the core of the Software and Electronic Engineering degree.

The project was therefore pivoted to a purely software-based solution. A mobile application combining machine learning-based visual tablet identification with a personalised tablet schedule management system. This approach addresses the core problem I am targeting, while making better use of the skills developed across the degree programme, particularly in areas of backend development, database design, and machine learning.

Several commercial products were reviewed during the research phase. Hardware dispensers including the TabTime Medelert, Pivotell Advance Dispenser, and Hero Smart Dispenser were examined, alongside software applications such as Smart Pill ID, Drugs.com, and Pharmasee. None of these solutions combined mobile-accessible, real-time image-based tablet classification with a personalised schedule and intake logging system in a single application.

The scope of the project encompasses the full software stack: a trained convolutional neural network for image classification, a RESTful API backend with secure authentication, a relational database, and a React Native mobile frontend. Each of these layers communicates to deliver the full user journey; Register an account, create a medication schedule, photograph or upload a tablet image, receive an identification, be informed whether the tablet is currently required, and log the intake.

The project was managed over two semesters of the 2025/2026 academic year, from September 2025 to April 2026, using Jira for sprint-based agile project management and OneNote for weekly development logging.

4 Research and Background

4.1 Machine Learning and Convolutional Neural Networks

Machine learning is the field of computer science concerned with developing systems that learn from data rather than following explicitly programmed rules. In the context of image classification, the dominant class of models is the Convolutional Neural Network (CNN). CNNs learn hierarchical representations of visual data, beginning with simple low-level features such as edges and gradually combining these into increasingly complex and class-specific representations as the input passes through successive layers.

The term 'convolutional' refers to the core mathematical operation performed within these networks. A convolution involves sliding a small matrix of learned weights, known as a kernel or filter, across the spatial dimensions of an input image. At each position, the filter performs an element-wise multiplication with the corresponding patch of the input and sums the results to produce a single output value. This process, repeated across all spatial positions, produces a feature map that highlights where a particular pattern is present in the image. By using many different filters in each layer, the network can extract a rich variety of features simultaneously.

The usefulness of this approach lies in the concept of local connectivity and weight sharing. Rather than connecting every input pixel to every neuron, each filter focuses on a small local region of the image. The same filter weights are reused across all spatial positions, reducing the number of learnable parameters dramatically. This also confers a degree of translational invariance: a filter that detects a horizontal edge will do so regardless of where in the image the edge appears.

4.2 CNN Layer Types

A typical CNN is composed of several distinct layer types, each serving a specific function:

Convolutional Layers extract feature maps from their input using learnable filters. In the earliest layers, filters tend to detect simple features such as edges and colour gradients. Deeper layers combine these to represent complex patterns such as shapes, textures, and eventually class-specific features.

Pooling Layers reduce the spatial dimensions of feature maps through a sub-sampling operation. The most common variant is max-pooling, which selects the maximum value within each non-overlapping pooling region. A 2x2 max-pooling operation reduces each spatial dimension by a factor of two, resulting in a 75% reduction in the number of values. This reduction serves two purposes: it decreases computational cost and introduces a degree of spatial invariance, making the network less sensitive to small shifts or distortions in the input.

Activation Functions introduce non-linearity into the network. Without non-linear activation functions, the composition of linear transformations across multiple layers would collapse to a single linear transformation, fundamentally limiting the network's expressive power. The Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, is the dominant choice in modern CNNs due to its simplicity and effectiveness in preventing the vanishing gradient problem.

Fully Connected Layers appear towards the end of the network. Every neuron in a fully connected layer receives input from every neuron in the preceding layer, enabling the network to integrate all detected features into a final decision. The output of the last fully connected layer is passed through a softmax activation to produce a probability distribution over the target classes.

Dropout is a regularisation technique frequently applied to fully connected layers during training. At each training step, a randomly selected subset of neurons are 'dropped' (their

outputs set to zero). This prevents co-adaptation between neurons and encourages the network to learn more robust, distributed representations, thereby reducing overfitting.

4.3 Training Deep Neural Networks

Training a CNN requires a labelled dataset. During training, the model processes each input, generates a prediction, and the loss function quantifies the discrepancy between the predicted output and the ground truth label. For multi-class classification tasks, categorical cross-entropy is the standard loss function. It computes the negative logarithm of the probability assigned to the correct class, penalising confident wrong predictions heavily.

Gradient descent is the optimisation algorithm used to minimise the loss. At each training step, the gradient of the loss with respect to each learnable parameter is computed via backpropagation, and the parameters are updated by a small amount in the direction that reduces the loss. The size of each update is controlled by the learning rate. Adaptive optimisers such as Adam (Adaptive Moment Estimation), or as I used, AdamW, maintain per-parameter learning rates adjusted based on the historical gradient magnitudes.

Overfitting is one of the central challenges in training deep networks. A model that has been trained for too many epochs, or on too small a dataset, may memorise specific training examples rather than learning general patterns. This manifests as a large gap between training accuracy (high) and validation accuracy (low). Overfitting was encountered repeatedly during the development of this project and addressed through several techniques: reducing training epochs, applying data augmentation, using the AdamW optimiser (which adds weight decay regularisation to Adam), reducing batch size, and ultimately switching to a pretrained model backbone.

Data augmentation artificially increases the effective size and diversity of the training dataset by applying random transformations to training images, such as horizontal and vertical flips, rotation, brightness and contrast adjustments, and zoom. Each augmented version of an image is treated as a distinct training sample. This encourages the model to learn features that are unchanging during these transformations.

4.4 Transfer Learning and ResNet50

Transfer learning is a machine learning paradigm in which a model previously trained on a large, general dataset is adapted for use on a new, more specific task. The key insight is that the lower layers of a CNN trained on a large image corpus such as ImageNet already encode general visual features such as edges, textures, colour gradients, and simple shapes that are valuable regardless of the specific classification task. Only the later, more task-specific layers need to be retrained.

This approach confers several practical advantages. Training a large CNN from scratch requires both a very large, labelled dataset and significant computational resources, which I discovered while trying to build my own CNN. Transfer learning allows high accuracy to be achieved with a much smaller task-specific dataset and in much less training time, since the general feature extraction capability is already learned. It also typically improves generalisation, since the backbone has been exposed to a far greater diversity of visual input during its original ImageNet training than would be feasible with a task-specific dataset.

ResNet50 (Residual Network, 50 layers) is a commonly used pretrained CNN architecture. Its defining innovation is the residual connection (also called a skip connection). Rather than passing the input through a sequence of layers sequentially, each residual block adds the original input directly to the output of the block. This shortcut allows gradients to flow directly through the network during backpropagation, effectively solving the vanishing gradient problem that makes training very deep networks difficult. As a result, very deep networks (50, 101, or more layers) can be trained reliably.

In this project, the ResNet50 model pretrained on ImageNet was loaded via the Keras API using the include_top=False parameter, which removes the original ImageNet classification head. A custom classification head consisting of a global average pooling layer followed by a dense softmax output layer for the four target classes was then appended. This approach I have adopted follows the standard transfer learning pattern for image classification with Keras.

An important preprocessing step when using ResNet50 is the application of the preprocess_input function from keras.applications.resnet50. This function normalises input pixel values to the scale expected by the ResNet50 model. Failing to apply this normalisation can significantly degrade classification performance, as I observed during development when prediction results did not match training accuracy.

4.5 Existing Similar Products

Prior to settling on the final project concept, a comprehensive review of existing products was conducted to understand the current state of the market and identify gaps the proposed application could address.

Three hardware-based automated pill dispensers were examined in detail:

- TabTime Medelert Automatic Pill Dispenser: A 28-slot removable tray dispenser operated by manually setting alarm times using three physical buttons. It uses AA batteries and can be filled at a pharmacy. While simple and reliable, it offers no machine learning, no mobile integration, and no intake logging.
- Pivotell Advance Dispenser: A tray-based dispenser with a self-locking shutter that rotates to present the correct compartment at the programmed alarm time. It supports up to 24 alarms per day and a maximum alarm duration of five hours. Like the TabTime, it has no mobile or software integration.
- Hero Smart Dispenser: The most sophisticated of the hardware products reviewed. It connects via Wi-Fi to a mobile app, supporting push reminders, missed-dose alerts, and remote monitoring of pill-taking status. Medication is loaded into containers and dispensed with a single button press according to a schedule configured in the app. This product was identified as the closest conceptual match to the project goal.

Three software-based tablet identification tools were also reviewed:

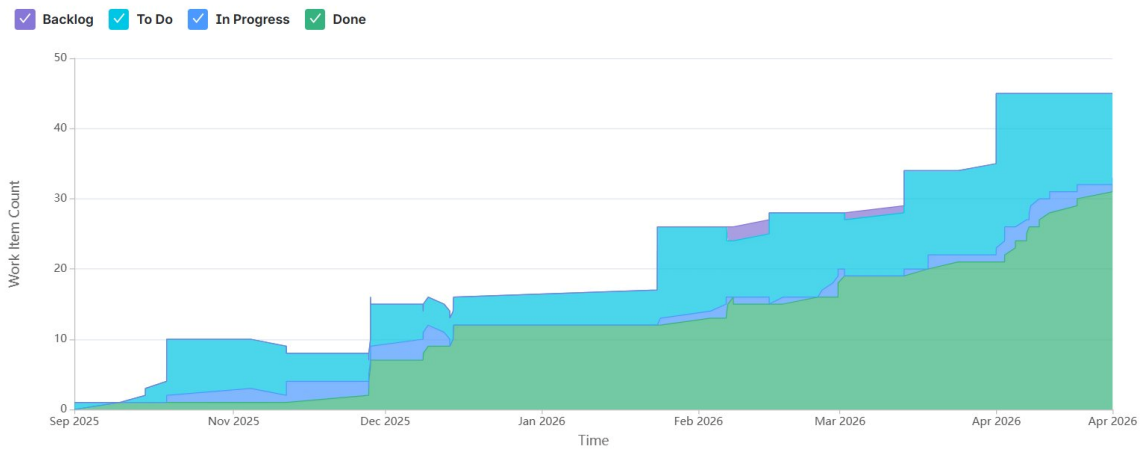
- Smart Pill ID: A mobile application that identifies tablets from photographs using physical attributes including inscription, colour, shape, and size. It also includes a barcode scanner for medication packaging. It does not offer schedule management or intake logging.
- Drugs.com: A web-based pill identification tool supporting manual attribute entry. It offers extensive drug information lookup but relies on manual input rather than computer vision.
- Pharmasee: An AI-powered pill recognition application targeted at medical professionals, capable of identifying unknown pills from a photograph. This is the most direct technical analogue to the ML component of the project but does not offer consumer-facing schedule management.

The key gap identified through this review was that no product combined real-time mobile image-based tablet identification with a personalised schedule check and intake logging in a single application designed for non-clinical users. This gap defines the unique value proposition of the Tablet Classification App.

5 Project Plan and Management

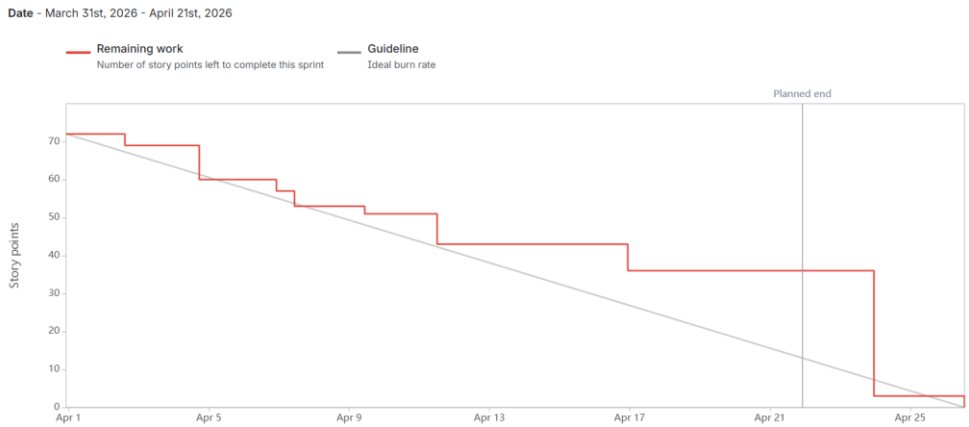
5.1 Methodology

The project was managed using an agile methodology. Jira was used as the project management tool, with work organised into two-week sprints up until the Christmas demonstrations, at which point, I switched to three-week sprints as I felt it helped me to better manage my time and stop pushing off certain tasks to the next sprint. Tasks were tracked on a Kanban-style board with columns representing To Do, In Progress, Done, On Hold, and Backlog. A cumulative flow diagram was generated from Jira throughout the project to provide a visual representation of work being added, progressed, and completed over time.

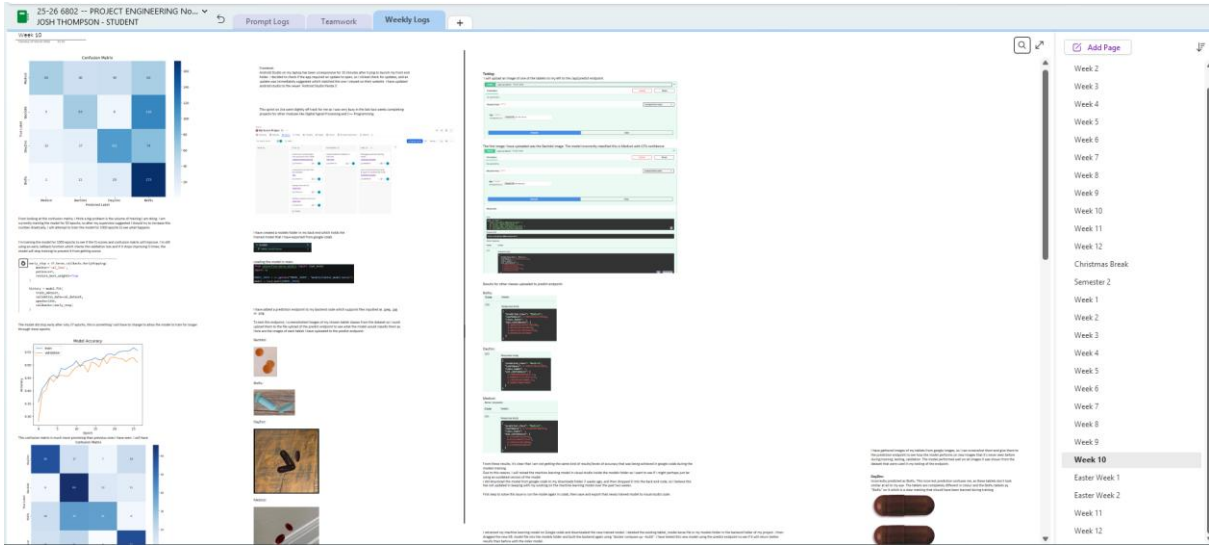


The SMART framework (Specific, Measurable, Achievable, Relevant, Time-bound) was applied when setting goals within each sprint, ensuring that targets were realistic given the available time and that progress could be objectively assessed.

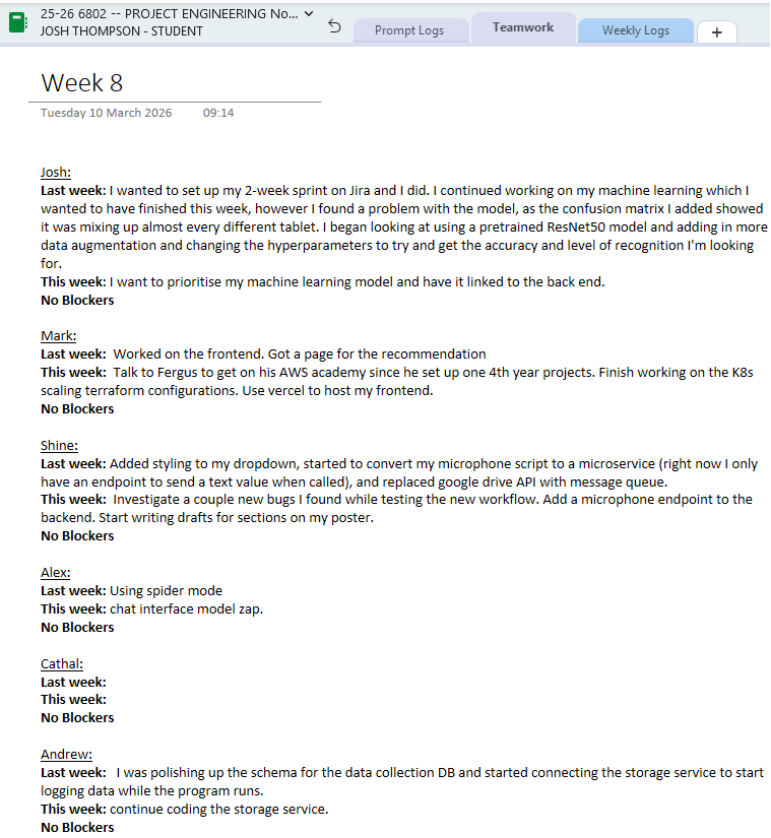
This is my sprint burndown chart. It shows the total work remaining for the current sprint, and the likelihood of the work being completed on time. It also shows a summary of scope changes (None in this sprint) and work done throughout the sprint. The red line shows how much work is remaining in the sprint, and the green line shows the ideal progress rate.



Weekly development logs were maintained throughout the project in Microsoft OneNote. These logs served as a reflective diary of decisions made, problems encountered, solutions found, and plans for the following week. They have been invaluable in reconstructing the development history for this report and in demonstrating the depth of work undertaken.



Logs of other classmate's work was kept also. These notes were recorded during Ben's Project Tutorial sessions. I completed standup meetings, informing my peers and Ben about what I had been doing each week. These meetings really helped me, as it was a way to keep me accountable for the work I had to complete each week, and keeping others accountable too for what they said they would do week-to-week.



5.2 Sprint Overview

The project spanned two academic semesters, from September 2025 to April 2026. The following table provides a high-level overview of the major milestones and work completed in each phase:

Period	Work Completed
Sept 2025 (Wk 1–2)	Project concept exploration. Hardware tablet dispenser concept examined across three design iterations. Decision to pivot to software-only ML approach following supervisor feedback.
Oct 2025 (Wk 3–5)	Research into existing pill dispensers and software identification tools. Project proposal submitted. Two software design ideas documented. Sprint 1 closed on Jira.
Nov 2025 (Wk 6–10)	Database comparison and selection of PostgreSQL. FastAPI backend development started. CRUD endpoints for users designed and implemented. PostgreSQL integration via SQLAlchemy. JWT research and implementation begun.
Dec 2025 (Wk 11–12)	JWT authentication completed. Bcrypt password length issue resolved. Docker port conflict resolved. Christmas demo preparation. Architecture diagram redesigned with larger fonts and technology labels.
Jan 2026 (Christmas / S2 Wk 1)	Christmas demo result 40%. Critical design flaw identified: login problematic for memory-impaired users. Face ID noted as future enhancement. Kaggle tablet dataset sourced and imported to Colab. Tablet schedule endpoint development.
Jan–Feb 2026 (S2 Wk 2–5)	CNN model training begun on Kaggle dataset. Initial training accuracy reasonable; validation accuracy poor due to overfitting. Epoch count and data split experimented with extensively.
Feb 2026 (S2 Wk 6)	Batch size reduced from 32 to 16 - validation accuracy improved by ~19%. AdamW optimiser adopted. ResNet50 research. React Native frontend development started.
Mar 2026 (S2 Wk 7–10)	ResNet50 transfer learning adopted. Layer freezing error discovered and corrected. Class reduction to 4 visually distinct tablets. Confusion matrix and F1 scores improved significantly. Prediction endpoint integrated into backend. Android Studio updated. Frontend camera integration.
Apr 2026 (Easter)	Full frontend restructure in JavaScript. Schedule stack navigator implemented. Expo image picker integrated. Intake logging added. API test suite developed with pytest. Poster completed. Report writing.

5.3 Challenges and Adaptations

Several significant challenges arose during the project that required the plan to change.

The first major challenge was the decision to move away from a hardware project. The initial concept was an automated tablet dispenser built with embedded hardware. Following supervisor guidance about the time cost of hardware construction and the limited scope for future software development, the project was redirected entirely towards a software application. This was the right decision I feel, as the resulting software project provided far more opportunities for me to apply the full range of skills I have acquired throughout my degree.

The second major challenge arose at the Christmas demonstration, when the lecturers identified a fundamental design contradiction: The application is designed to help people

with memory difficulties, yet it requires them to remember a username and password to log in. This was a significant oversight that had not been considered during design. The immediate response was to acknowledge the issue and propose biometric login (Face ID) as a future enhancement. My supervisor advised that Face ID should be deprioritised for the remainder of the project in favour of completing core functionality.

The third and most persistent challenge was the machine learning model development. Training a CNN that generalises well to unseen images took far longer than anticipated, requiring approximately ten weeks of iterative experimentation. Key difficulties included overfitting, dataset quality issues, and class confusion caused by the visual similarity of many of the ten tablet types in the Kaggle dataset. The eventual resolution - adopting ResNet50 transfer learning and reducing to four visually distinct classes This decision was not arrived at immediately but emerged through a systematic process of experimentation and reflection.

A fourth challenge was encountered during frontend development, where the initial Expo project template was generated using TypeScript, a language I am not familiar with. This required restarting the frontend from scratch using a JavaScript template, which cost some development time but ultimately produced a more manageable codebase.

A fifth challenge was network connectivity during Expo Go development. When working from home, the frontend was configured with the local network IP address. When returning to college accommodation with a different network, the IP address had changed and the Expo Go app could not connect. The issue was further compounded by the local network firewall treating the phone hotspot as a public network. The resolution was to switch from a public to a private network configuration in Norton’s advanced network security and firewall settings, which resolved the firewall restriction.

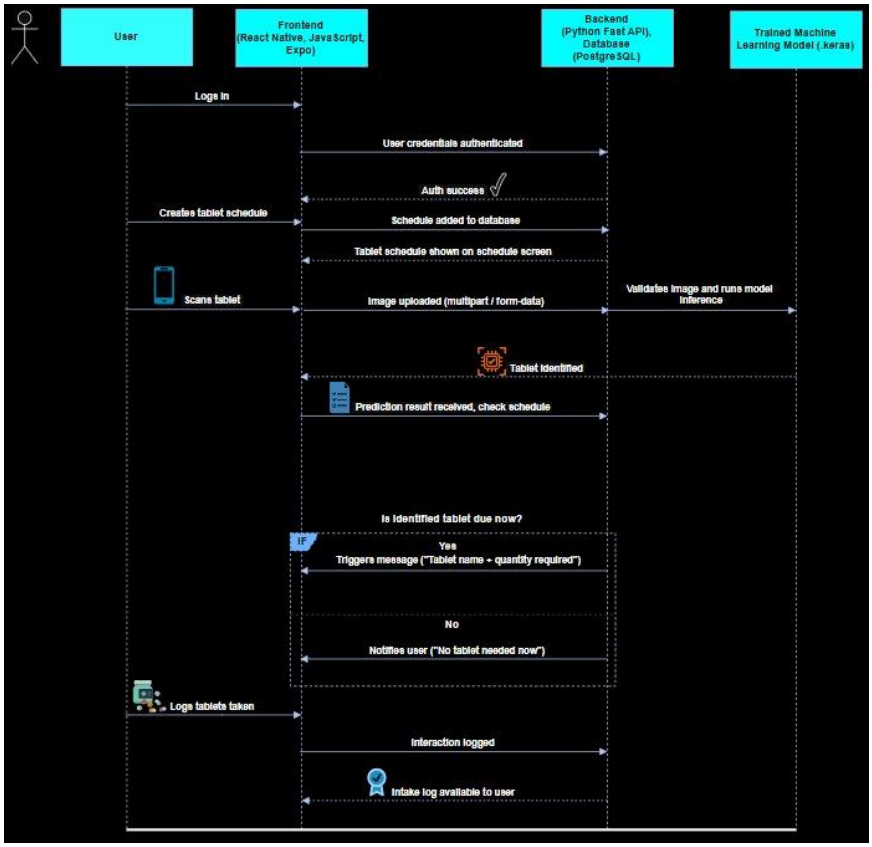
6 System Architecture

6.1 Overall Architecture

The system follows a layered client-server architecture comprising four distinct components: a mobile frontend, a backend API, a relational database, and a machine learning model. Each component has a single, well-defined responsibility, and the interaction between components is governed by clearly defined interfaces.

The mobile frontend communicates with the backend exclusively over HTTP, using JSON for data exchange and multipart form data for image uploads. The backend exposes a set of RESTful endpoints, handles authentication, queries the database, and invokes the machine learning model for prediction. The database provides persistent storage for all user and schedule data. The machine learning model is loaded into backend memory at startup and invoked synchronously within the prediction endpoint.

The backend services are containerised using Docker, with a docker-compose.yml file defining two services: a PostgreSQL database container and the FastAPI application container. The FastAPI container depends on the database container being healthy before starting, ensuring correct startup ordering. Environment variables are used to configure database connection strings, JWT secret keys, and other environment-specific settings, keeping sensitive configuration out of the codebase.



6.2 Frontend

The frontend is a React Native mobile application built using the Expo framework, written entirely in JavaScript. React Native was chosen because it enables cross-platform mobile development from a single codebase, targeting both Android and iOS devices. Expo was selected as the development framework on top of React Native because I have used it in labs for another module (Mobile App Development) so have become familiar with it, also, it provides a large library of pre-built modules for device hardware access, and allows live

development on a physical device using the Expo Go application without requiring a full native build toolchain.

The application uses the React Navigation library for screen management. The overall navigation structure consists of a stack navigator (Home, Schedules, and Tablet Prediction) at the top level, with another embedded stack navigator used within the Schedules section to manage the navigation between the schedule home, view, and create screens.

State management is handled using React's built-in useState and useEffect hooks. Authenticated user state (specifically the JWT access token) is maintained in React state and passed through the component tree as needed. The expo-image-picker package provides access to the device camera and photo library for the tablet scanning feature.

The backend API base URL is configured in a config.js file. During development, this required manual updating when my laptop's local network changed (as documented in the weekly logs). In a production deployment, this would be replaced with a persistent public URL.

6.3 Backend

The backend is a Python FastAPI application served using Uvicorn. FastAPI was selected as I have experience using it for another module (Continuous Integration & Continuous Delivery 2). It offers high performance for an I/O-bound API workload, generates interactive API documentation (Swagger UI) automatically from the type-annotated Python code, and integrates naturally with Pydantic for request and response validation.

The application is structured into separate modules for database models, Pydantic schemas, CRUD operations, authentication utilities, and route handlers. This separation of concerns makes the codebase easier to navigate, test, and extend. The main application file (main.py) assembles the application, loads the machine learning model at startup, and registers all route handlers.

The backend exposes the following endpoint groups:

- /api/users - User registration and retrieval
- /api/login - User authentication and JWT issuance
- /api/tablet-schedules - CRUD operations for tablet schedules
- /api/log-intake - Intake logging (POST and GET)
- /api/predict - Tablet image classification

All endpoints except registration and login are protected by JWT authentication. A FastAPI dependency is used to extract and validate the JWT from the Authorization header of incoming requests, returning the authenticated user's ID for use within the endpoint logic.

6.4 Database

PostgreSQL was selected as the database management system after a comparative analysis of MySQL, PostgreSQL, and MongoDB. The comparison I have documented in the Week 6 development log, evaluated each system across criteria including database type, ACID compliance, complex query support, horizontal scalability, and suitability for the use case.

MySQL and PostgreSQL are both relational database management systems, while MongoDB is a document-oriented NoSQL database. For this project, a relational model is the natural fit: the application needs to link user accounts to their tablet schedules (a one-to-many relationship) and to their intake logs (another one-to-many relationship), and relational databases are specifically designed to manage these kinds of structured, interdependent records efficiently.

PostgreSQL was preferred over MySQL primarily on the grounds of its full ACID compliance and superior support for complex queries and data integrity constraints. These properties are especially important in a medical context where data accuracy is critical. I also had recent practical experience with PostgreSQL from the CICD2 module, which reduced the setup time.

The database schema comprises three tables:

- users: user_id (integer, primary key, auto-increment), name (varchar), email (varchar, unique), password_hash (varchar)
- tablet_schedules: id (integer, primary key), user_id (integer, foreign key to users.user_id), medication_name (varchar), dosage (varchar), times (array/list of time-of-day values)
- intake_logs: id (integer, primary key), user_id (integer, foreign key to users.user_id), medication_name (varchar), taken_at (timestamp), taken (boolean)

Database interactions are handled through SQLAlchemy as the ORM layer. SQLAlchemy models define the database schema in Python, and all interactions (inserts, queries, updates, deletes) are expressed using the SQLAlchemy session API. This abstracts the raw SQL and allows the application to be developed against a consistent Python interface regardless of the underlying database engine.

6.5 Machine Learning Model

The machine learning model is a Keras deep learning classification model based on the pretrained ResNet50 architecture. It classifies input images into one of four tablet classes: Bactidol, Bioflu, DayZinc, and Medicol. The model was developed and trained in a Google Colab notebook and exported as a .keras file.

At backend startup, the .keras file is loaded into memory using the keras.models.load_model function. This load-at-startup pattern ensures the model is ready to serve requests immediately and avoids the latency of loading the model on each prediction request.

The prediction endpoint accepts an uploaded image file, performs the following preprocessing steps, and then runs inference:

- The image is opened using the Python Imaging Library (PIL) and converted to RGB mode to ensure consistent three-channel input.
- The image is resized to 224x224 pixels, which is the input dimension expected by ResNet50.
- The pixel array is converted to a NumPy array and expanded to include a batch dimension (shape: [1, 224, 224, 3]).
- The ResNet50 preprocess_input function is applied to normalise the pixel values to the scale used during ResNet50 pretraining.
- The preprocessed array is passed to the model for inference, returning a probability distribution over the four classes.
- The class with the highest probability is selected and returned to the frontend as the predicted label, along with its confidence score.

7 Implementation

7.1 Backend and API Development

The backend was developed iteratively over several sprints across Semester 1, beginning in November 2025. The initial structure established CRUD endpoints for user management following standard REST conventions: POST endpoints for creation, GET endpoints for retrieval, PUT endpoints for updates, and DELETE endpoints for removal.

The application is containerised using Docker. A Dockerfile installs all Python dependencies from a requirements.txt file and starts the Uvicorn server on port 8005. A docker-compose.yml file defines two services: the PostgreSQL database and the FastAPI application. The FastAPI service uses a healthcheck-based depends_on configuration to ensure the database container is fully initialised before the API container starts. This prevents startup race conditions that would cause connection failures.

During development, the Swagger UI automatically generated by FastAPI at the /docs endpoint was the primary tool for testing API endpoints. This interface allows HTTP requests to be constructed and executed directly from the browser, displaying full request and response details. Later in development, Postman was also used for more complex testing scenarios requiring persistent headers and saved requests.

A Docker port conflict was encountered during development (documented in the Week 11 log) in which three containers from previous sessions were still running and occupying ports. This prevented new containers from binding to the required ports, resulting in an internal server error when attempting to create users. The resolution was to stop and remove all running containers in Docker Desktop before rebuilding.

7.2 Authentication and JWT

User authentication is implemented using JSON Web Tokens (JWTs) via the python-jose library. JWTs are a compact, URL-safe mechanism for representing authentication claims between two parties, in this case between the mobile frontend and the FastAPI backend. A JWT consists of three base64-encoded sections separated by dots: a header specifying the token type and signing algorithm, a payload containing the user ID and token expiry time, and a cryptographic signature that allows the backend to verify the token has not been tampered with.

When a user successfully logs in by submitting their email and password to the /api/login endpoint, the backend verifies the password hash, generates a JWT signed with a secret key, and returns the token to the frontend. The frontend stores this token and includes it in the Authorization header of all subsequent requests to protected endpoints, using the Bearer token scheme. A FastAPI dependency function extracts and validates the token on each protected request, returning the authenticated user's ID.

The choice of JWTs over traditional session-based authentication was made on the grounds of statelessness (the server does not need to store session data), ease of scaling across multiple server instances, and suitability for mobile clients. The key disadvantage of JWTs is that individual tokens cannot be revoked before they expire without maintaining a server-side blacklist was acknowledged but accepted given the development context and time constraints.

Passwords are hashed using bcrypt via the passlib library. Bcrypt is a deliberately slow hashing algorithm, making brute-force attacks computationally expensive. Plain-text passwords are never stored or logged. A known limitation of bcrypt is that it silently truncates passwords longer than 72 bytes. This was encountered during development when an error was thrown rather than silent truncation. The issue was resolved by explicitly truncating the password to 72 bytes before passing it to the bcrypt hash function.

A significant design issue was raised at the Christmas demonstration: requiring memory-impaired users to remember a login password contradicts the accessibility goals of the application. The proposed future resolution of biometric Face ID login using the device's fingerprint or facial recognition capability was acknowledged and added to the backlog. The Expo framework provides the expo-local-authentication package for this purpose, but implementation was deferred to keep the focus on core functionality.

7.3 Tablet Schedule Endpoints

The tablet schedule feature allows authenticated users to create, view, update, and delete their medication schedules. A schedule entry associates a medication name, dosage string, and one or more time-of-day values (morning, afternoon, or evening) with the authenticated user's account.

The POST /api/tablet-schedules endpoint accepts a JSON body containing the medication name, dosage, and time list. The data is validated by a Pydantic schema before being passed to the database layer. A corresponding GET /api/tablet-schedules endpoint returns all schedules associated with the authenticated user. DELETE and PUT endpoints were also implemented to allow individual schedules to be removed or updated.

On the frontend, the schedules section is structured using a stack navigator providing three screens: a Schedule Home screen with buttons navigating to the Create and View screens, a Create Schedule screen with input fields for medication name, dosage, and time of day, and a View My Schedule screen that fetches and displays the user's schedules from the backend, grouped by time of day (morning, afternoon, evening).

An issue encountered during the schedule endpoint development was a validation error in Swagger when testing the Create Schedule endpoint. The times field was being passed as an integer input type rather than accepting a list of strings. This required adjustment of the Pydantic schema to correctly declare the times field as a List[str] type.

7.4 Machine Learning Model Development

The machine learning model was developed through an extended iterative process spanning approximately ten weeks of the second semester. The following subsections document each distinct phase of development.

7.4.1 Dataset Selection

An initial attempt was made to build a custom dataset by photographing physical tablets in a controlled environment. A desk was set up with a white paper background and a desk lamp positioned directly overhead to minimise shadows. Sixteen photographs were taken of each of three tablet types from various angles and distances. These images were then transferred to a local folder and manually sorted into training, testing, and validation subsets.

However, this approach quickly proved problematic. Even with sixteen images per tablet, the resulting dataset was far too small to train a generalisable model. More critically, the process of capturing, reviewing, and cleaning a sufficient quantity of high-quality images was extremely time-consuming, and the resulting images lacked the visual variety that would allow the model to generalise to real-world conditions.

Following supervisor guidance, a publicly available dataset was sourced from Kaggle: the 'Pharmaceutical Drugs and Vitamins Synthetic Images' dataset. This dataset comprises 10,000 images across 10 tablet classes (1,000 images per class). The images are synthetic but visually realistic and vary in background, lighting, and tablet orientation, providing good training diversity. The Kaggle dataset was imported directly into a Google Colab notebook and split into training, validation, and testing subsets.

7.4.2 Initial CNN Development

An initial CNN was trained from scratch using a standard architecture. The network architecture followed the classical pattern of alternating convolutional and max-pooling blocks, followed by a global average pooling layer and a dense output layer with softmax activation. The model was compiled with the Adam optimiser and categorical cross-entropy loss, and accuracy was monitored as the primary metric.

Training for 10 epochs on the full 10-class dataset produced a training accuracy that improved steadily across epochs, but validation accuracy remained substantially lower, indicating overfitting. The training accuracy/loss and validation accuracy/loss curves were plotted using Matplotlib after each training run to visualise the generalisation gap.

7.4.3 Hyperparameter Tuning

An extensive period of hyperparameter experimentation followed the initial training run. The key variables adjusted were:

- Number of epochs: Experimented with values of 2, 10, 15, 25, and 30. Training for 2 epochs produced very low accuracy for both training and validation. Training for 30 epochs produced 97.75% training accuracy but only 51.22% validation accuracy which I found to be a clear indicator of severe overfitting. Training for 15 epochs represented a better compromise, with 96.6% training accuracy and higher validation accuracy than the 30-epoch run.
- Data split ratio: The default split of 80/10/10 (training/validation/testing) was adjusted to 70/15/15, then 65/25/10. A colleague suggested making the testing split larger than the validation split; this was implemented with a 65% training / 25% testing / 10% validation split. This change did not substantially improve validation accuracy.
- Batch size: Reducing the batch size from 32 to 16 produced the single largest accuracy improvement observed during the tuning phase, in an increase in validation accuracy of approximately 19 percentage points. Smaller batch sizes introduce more noise into the gradient estimates, which can act as an implicit regulariser and improve generalisation. The improvement was large enough to retain the batch size of 16 for all subsequent training runs.
- Optimiser: The AdamW optimiser was substituted for Adam. AdamW adds weight decay (L2 regularisation) to Adam's update rule, penalising large parameter values and further reducing overfitting. This provided a modest additional improvement in validation accuracy.

Despite these improvements, when examining the per-class precision, recall, and F1 scores, the model continued to show low F1 scores for many of the 10 classes. Examination of the confusion matrix revealed that the model was systematically confusing certain tablet classes with others. The most problematic class was 'Kremil S', which was heavily over-predicted. The model frequently classified images of other tablets as Kremil S. This suggested that either the dataset images for Kremil S were not sufficiently distinctive, or that there was a dataset quality issue.

7.4.4 Class Reduction and Visualisation

To address the persistent class confusion, the decision was made to reduce the number of classes from 10 to 4. The rationale was that the fundamental problem was the visual similarity between many of the tablets in the dataset. By selecting only four tablets that were maximally visually distinct from one another, the classification task could be made tractable for the available dataset size.

To support this decision, all 10 tablet classes in the dataset were visually inspected by looking at representative images of each class. The key distinguishing visual features assessed were shape (round vs oval vs capsule), colour, and any surface markings or text. Based on this inspection, the following four classes were selected as the most visually distinct:

- Bactidol: Consistent orange-amber colour with no strong markings. Round and distinctive in colour compared to all other selected classes.
- Bioflu: Blue oblong tablet with the text 'Bioflu' clearly printed on the surface. Distinctive both in colour and in its text marking.
- DayZinc: Small dark brown/black oval tablet. Distinctive in its dark colour and small size.
- Medicol: Large red oval tablet. Distinctive in its bold red colour and large size.

The class 'Kremil S' was explicitly excluded based on its history of being the most over-predicted class in earlier runs. After re-training with only these four classes, the confusion matrix was markedly cleaner, and the per-class F1 scores improved substantially.

7.4.5 ResNet50 Transfer Learning

Despite the improvement from class reduction, the model trained from scratch still showed limitations when tested on entirely unseen images sourced from the internet (i.e., not from the Kaggle dataset). Several of these real-world images were misclassified, suggesting the model had learned to identify specific visual characteristics of the synthetic dataset images rather than the true distinguishing features of the tablet classes.

Transfer learning with a pretrained ResNet50 backbone was adopted to address this. The ResNet50 model pretrained on ImageNet was loaded via keras.applications.ResNet50 with include_top=False, removing the original 1000-class classification head. A custom head was added on top of the ResNet50 feature extraction layers, consisting of a GlobalAveragePooling2D layer and a Dense(4, activation='softmax') output layer.

A critical error was discovered during this phase: an initial implementation froze too many ResNet50 layers by setting layer.trainable = False for all layers. This prevented the feature extraction layers from adapting to the tablet classification task. After reviewing the code and consulting with an AI assistant, the error was identified and corrected by setting all layers to trainable=True. The corrected model then underwent full fine-tuning during training.

Additionally, the preprocess_input function from keras.applications.resnet50 was not being applied to image batches in the initial ResNet50 implementation. This was identified as a likely cause of the discrepancy between training metrics and real-world performance, as the model was being trained on improperly normalised data. After applying preprocess_input to all training, validation, and test batches (as recommended in a Stack Overflow discussion), model performance improved significantly.

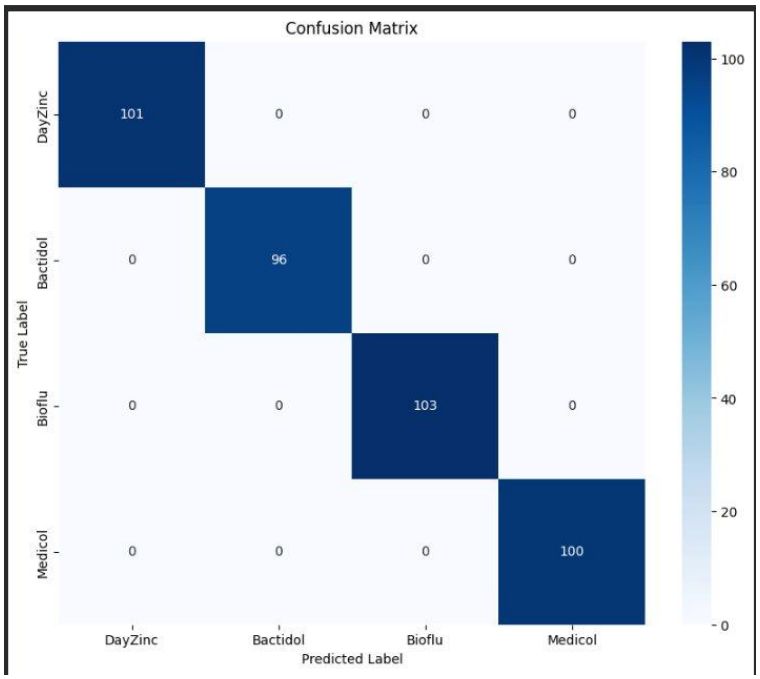
7.4.6 Final Model Configuration

The final model configuration that was exported for use in the backend was trained with the following settings:

- Architecture: ResNet50 pretrained backbone (all layers trainable) with GlobalAveragePooling2D and Dense(4, softmax) head
- Optimiser: AdamW with weight decay 0.0005
- Loss function: Categorical cross-entropy
- Batch size: 16
- Training epoch target: 1000, with early stopping monitoring validation loss with patience=5
- Data split: 65% training / 25% testing / 10% validation, using the four selected classes
- Preprocessing: ResNet50 preprocess_input applied to all batches

The early stopping callback halted training after 27 epochs when validation loss ceased to improve. The resulting confusion matrix showed the majority of predictions on the diagonal,

indicating predominantly correct classification across all four classes. The F1 scores for the final model run showed strong performance for all four classes.



Precision/Recall/F1 Scores

```
print(classification_report(
    true_labels,
    predicted_labels,
    target_names=class_names
))
```

```
              precision    recall  f1-score   support

     DayZinc       1.00      1.00      1.00       101
    Bactidol       1.00      1.00      1.00        96
      Bioflu       1.00      1.00      1.00       103
     Medicol       1.00      1.00      1.00       100

    accuracy                           1.00       400
   macro avg       1.00      1.00      1.00       400
weighted avg       1.00      1.00      1.00       400
```

7.5 Frontend Development

The frontend was built using React Native with the Expo framework in JavaScript. An initial attempt was made using a TypeScript Expo project template, but this was abandoned because TypeScript is not covered in the degree programme and the unfamiliar type system introduced significant friction during development. The project was restarted using a JavaScript template, which allowed progress to be made more efficiently.

The app navigation structure is built with React Navigation. A stack navigator provides top-level navigation between the Home, Schedules, and Tablet Prediction screens using buttons. Within the Schedules screen, a stack navigator manages three screens: the Schedule Home screen (with navigation buttons), the View My Schedule screen, and the Create Schedule screen.

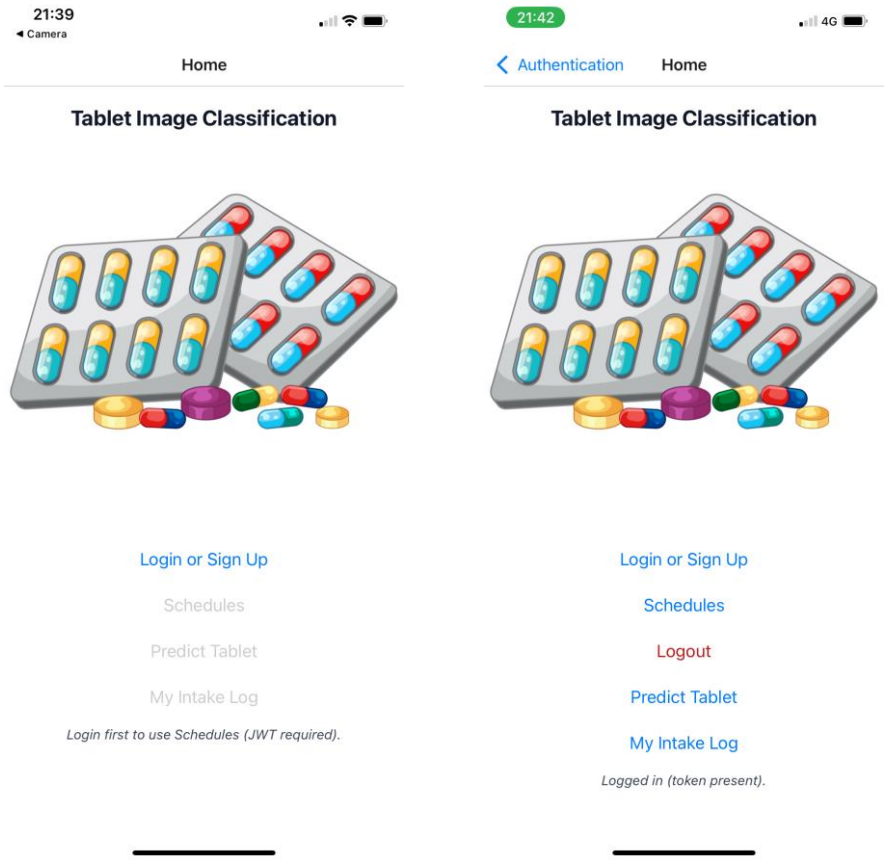
The Home screen serves as the main landing page after login. It displays a title and provides navigation links to all key features of the application. The screen also provides useful messages, such as prompting the user to log in before accessing schedule features.

The Authentication screen supports both user registration and login. It contains input fields for name (only required for registration), email, and password, along with Register and Login buttons. On successful registration, a new user record is created in the database via the POST /api/users endpoint. On successful login, the JWT access token is returned and stored in React state for use in authenticated requests.

The Create Schedule screen allows the user to enter a medication name, dosage, and time of day, and submit this to the POST /api/tablet-schedules endpoint. The View My Schedule screen fetches the user's schedules using the GET /api/tablet-schedules endpoint and displays them grouped into Morning, Afternoon, and Evening sections.

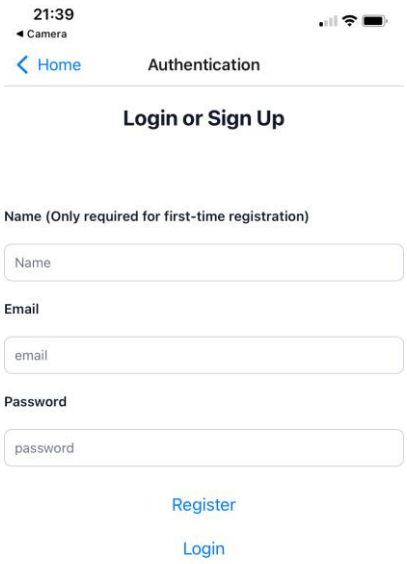
A persistent issue during frontend development was Expo Go connectivity. The application communicates with the backend using the host machine's local network IP address. When my network changed (between home in Wicklow and accommodation in Galway), the IP address changed and the Expo Go app could not connect. The issue was discovered when using a mobile phone hotspot, as Norton on my laptop classified the hotspot network as a Public Network and applied stricter firewall rules that blocked the connection. The resolution was to change the hotspot network classification from Public to Private in Norton's Security, Advanced, Smart Firewall settings, allowing Expo Metro Bundler traffic through.

Figure 1: Home Screen (before and after login)



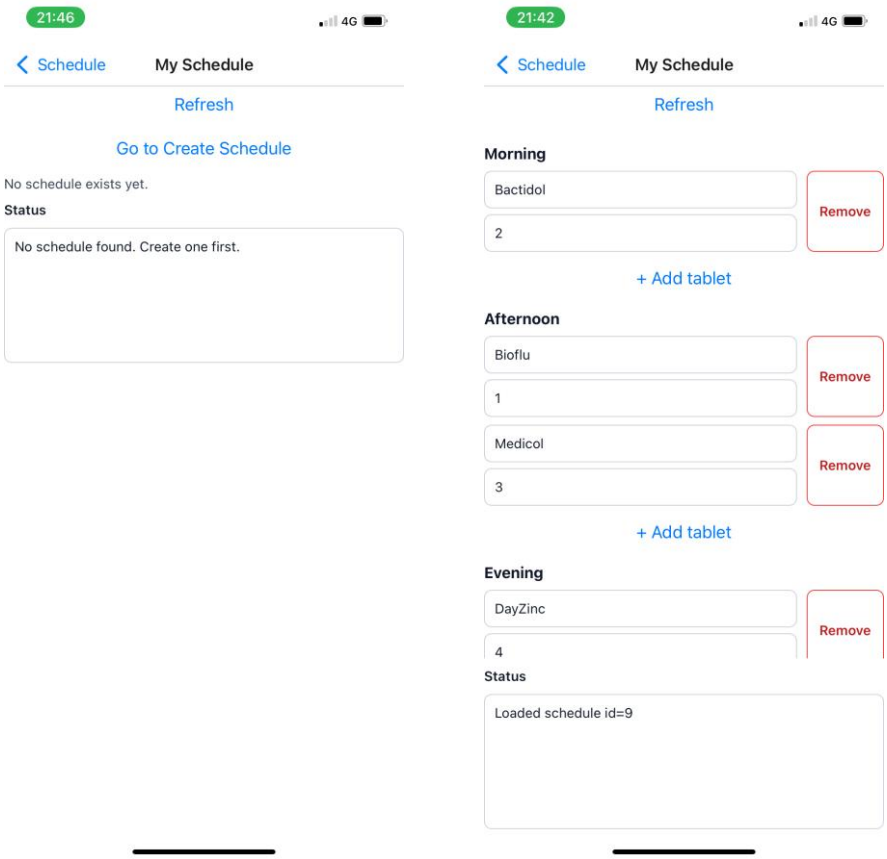
Left: Home screen before login (features greyed out). Right: Home screen after login (all features accessible).

Figure 2: Authentication Screen

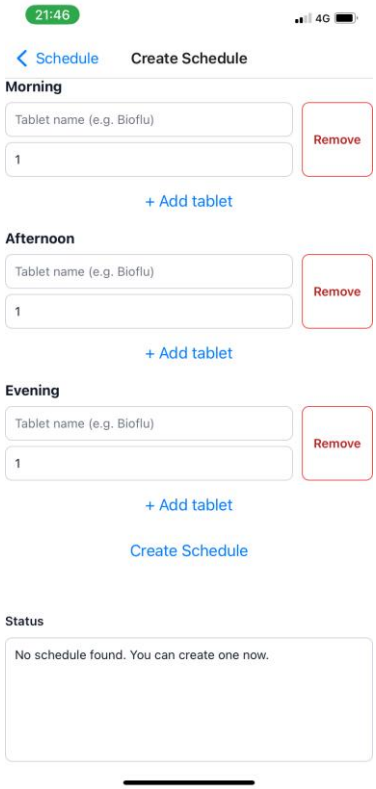


Authentication screen supporting both registration and login with email and password.

Figure 3: Schedule Screens

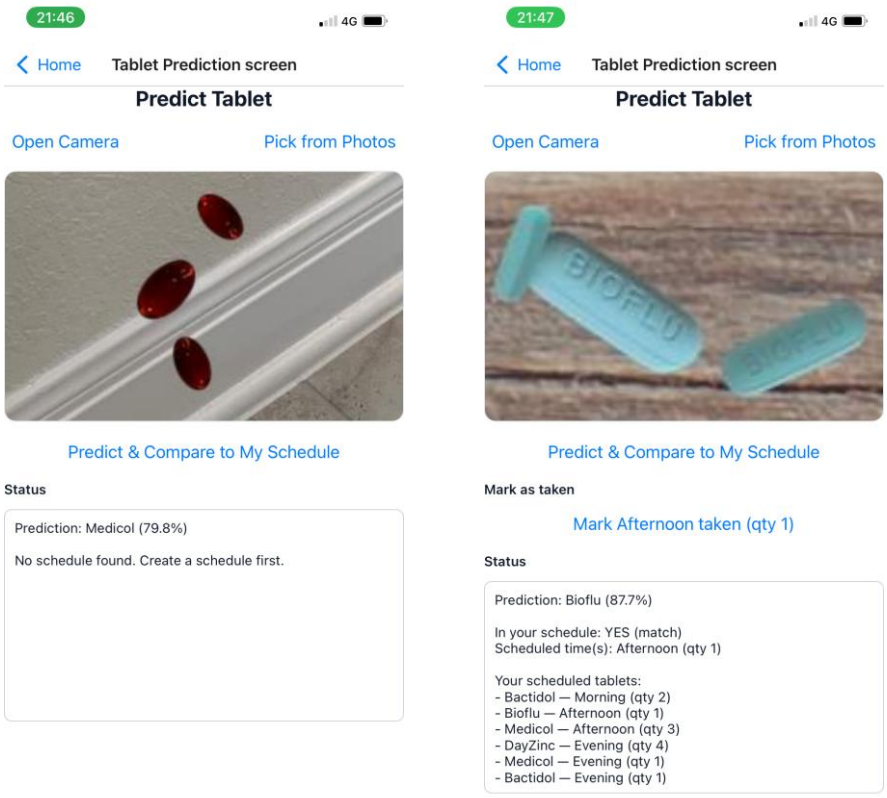


Left: View schedule screen with no schedule created. Right: View schedule screen populated with tablets across Morning, Afternoon and Evening.



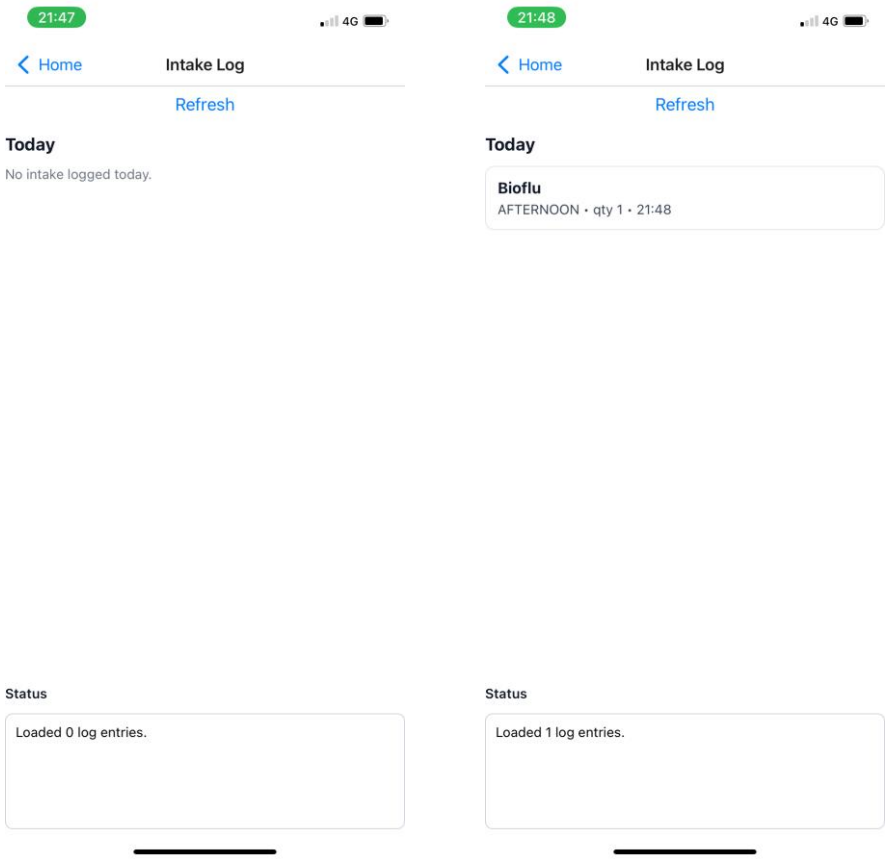
Create schedule screen allowing the user to add tablet names and quantities per time of day.

Figure 4: Tablet Prediction Screen



Left: Prediction result (Medicol, 79.8%) with no schedule created. Right: Prediction result (Bioflu, 87.7%) matching the user's afternoon schedule, with Mark as Taken button.

Figure 5: Intake Log Screen



Left: Intake log with no entries. Right: Intake log showing Bioflu logged for the Afternoon session.

7.6 Prediction Endpoint Integration

The Tablet Prediction screen is the core feature of the application. It allows the user to select a tablet image from their device's photo library or capture a new photograph using the device camera, send the image to the backend for classification, and receive an identification and schedule comparison.

The expo-image-picker package is used to implement the image selection functionality. This package provides a cross-platform API for accessing both the camera and photo library. When the user presses the 'Select Image' button, the image picker launches and the user selects or captures an image. The selected image is stored in component state as a URI, and a preview is displayed on screen.

When the user presses 'Predict and Compare to My Schedule', the image is sent to the backend /api/predict endpoint as a multipart/form-data POST request. The backend preprocesses the image, runs inference through the loaded ResNet50 model, and returns the predicted class label and confidence score. The frontend receives this response and queries the user's schedule to determine whether the identified tablet is required at the current time of day. The result is displayed on screen: if the tablet is in the schedule for the current time period, the tablet name, dosage, and required quantity are shown; if not, a 'No tablet needed now' message is displayed.

During integration testing, it was found that the prediction endpoint was returning incorrect results when tested with the exported model file. Investigation revealed that the .keras model file in the backend had not been updated to reflect the latest training run. Replacing the model file with the most recently exported version resolved the issue. This highlighted the importance of ensuring the backend model file is always kept in sync with the most recent Colab training output.

7.7 Intake Logging

Once the user has reviewed their prediction and schedule comparison, they can log that they have taken their tablet by pressing a 'Mark as Taken' button. This sends a POST request to the /api/log-intake endpoint with the medication name, the current timestamp, and the user's ID (extracted from the JWT). The backend records this intake event in the intake_logs table.

A GET /api/log-intake endpoint allows the user to retrieve their historical intake log, providing a record of medication adherence over time. This log can be reviewed by the user or a caregiver to identify missed doses or patterns of non-adherence. The frontend displays today's intake log on the prediction result screen.

7.8 API Testing

A pytest-based API test suite was developed during the Easter development period to formally verify the backend endpoints. The test suite is structured as follows:

A conftest.py configuration file provides shared test fixtures. I had originally used monkeypatch to substitute a 'dummy' machine learning model for the real Keras model during testing, however, I then moved to unittest.mock.patch as the patch needed to be applied before pytest fixtures are available - if patched too late, the real model would already have been loaded. This is important because loading a TensorFlow model is slow, the model file may not be present in all test environments, and the API tests are concerned with endpoint logic and data correctness rather than ML model accuracy (which I have evaluated separately in Colab). The test environment is configured by setting environment variables directly in conftest.py at module level, overriding the application's database URL with a file-based SQLite database (sqlite:///./test_temp.db). This ensures tests do not affect the development PostgreSQL database and can be run in any environment without requiring Docker or a running PostgreSQL instance.

The test_api_flows.py file contains the following tests:

Test Name	What it Verifies
`test_signup_then_login_returns_token`	Registering a new user and logging in returns a valid JWT access token
`test_predicted_route_requires_token`	Accessing the schedules endpoint without a token returns HTTP 401 Unauthorized
`test_create_and_list_schedules`	Creating a tablet schedule and retrieving it returns the correct data
`test_update_and_delete_schedule`	Updating and then deleting a schedule works correctly
`test_create_intake_and_get_today`	Logging a tablet intake and retrieving today's log returns the correct record
`test_predict_accepts_image_and_returns_probs`	Uploading an image to the predict endpoint returns a response with the correct structure (prediction label and confidence)
`test_get_user_not_found`	Get a user by ID that doesn’t exist
`test_delete_user`	Deleting a user by id removes them from the database
`test_login_wrong_password`	Login with wrong password returns correct error message (401)

`test_login_wrong_email`	Login with wrong email returns correct error message (401)
`test_duplicate_user_registration`	Same email registered twice should create a conflict (409)
`test_predict_invalid_file_type`	Endpoint won't except non-image file types (.jpg, .jpeg, .png)
`test_schedule_not_found`	Cannot update a schedule if a schedule is not found
`test_delete_schedule_not_found`	Cannot delete a schedule if the schedule is not found
`test_get_all_users`	Authenticated user can return a list of all users (This is not included in the frontend, for privacy reasons)

8 Results and Testing

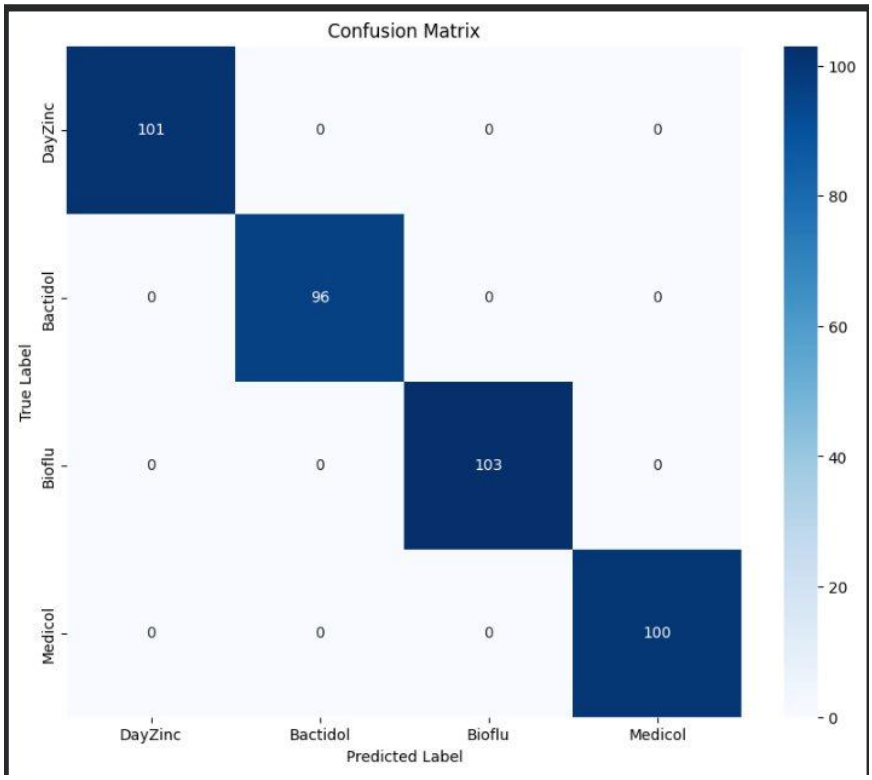
8.1 Machine Learning Results

The machine learning model was evaluated using confusion matrices and per-class classification reports (precision, recall, and F1 score) generated in Google Colab after each training run. The final model demonstrated substantially improved performance compared to earlier iterations.

Precision measures the proportion of predictions for a given class that were correct: of all the images the model classified as Bactidol, what fraction were actually Bactidol? Recall measures the proportion of actual instances of a class that were correctly identified: of all actual Bactidol images in the test set, what fraction did the model correctly identify? The F1 score is the harmonic mean of precision and recall, providing a single summary metric that balances both.

The progression of model performance across the key development phases is summarised below:

Configuration	Training Accuracy	Validation Accuracy
Scratch CNN, 10 classes, 10 epochs, batch=32	~85%	~42%
Scratch CNN, 4 classes, batch=32	~97%	~64%
Scratch CNN, 4 classes, batch=16 (key change)	~98%	~83%
ResNet50, 4 classes, AdamW, early stopping	>99%	>95% (F1: 1.00 all classes)



Precision/Recall/F1 Scores

```
print(classification_report(
    true_labels,
    predicted_labels,
    target_names=class_names
))
```

```
              precision    recall  f1-score   support

     DayZinc       1.00      1.00      1.00       101
    Bactidol       1.00      1.00      1.00        96
      Bioflu       1.00      1.00      1.00       103
     Medicol       1.00      1.00      1.00       100

    accuracy                           1.00       400
   macro avg       1.00      1.00      1.00       400
weighted avg       1.00      1.00      1.00       400
```

The confusion matrix from the final ResNet50 model run showed predictions concentrated heavily on the diagonal, with minimal off-diagonal entries, indicating that the model was rarely confusing one class for another. Bactidol was the most reliably classified tablet, consistent with it having the most visually distinctive appearance. Bioflu and Medicol were also classified with high reliability. DayZinc showed the most variability in earlier models but improved with the ResNet50 backbone.

8.2 API Test Results

The pytest test suite was run against the FastAPI backend using a file-based SQLite test database and the dummy ML model. All 15 tests passed successfully, confirming the correctness of the endpoint logic for authentication, schedule management, intake logging, and prediction response structure.

```
PS C:\Users\joshe\OneDrive - Atlantic TU\Project Engineering\FinalYearProject-3> python -m pytest tests/ --cov=app --cov-report=term-missing -v
========================================================================================== test session starts ==================================
================
platform win32 -- Python 3.13.1, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\joshe\OneDrive - Atlantic TU\Project Engineering\FinalYearProject-3
configfile: pytest.ini
plugins: anyio-4.11.0, cov-7.1.0
collected 15 items


tests\test_api_flows.py ...............
                                        [100%]

========================================================================================== tests coverage ==================================
================
_____________________________________________________________________________ coverage: platform win32, python 3.13.1-final-0 _____________________
_________________

Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
app\__init__.py       0      0   100%
app\database.py      26      6    77%   30-31, 37-41
app\main.py         177     17    90%   32-33, 98, 100-101, 105, 152, 167, 172-173, 187-188, 194, 241, 246, 248, 260
app\models.py        33      0   100%
app\schemas.py       53      0   100%
-----------------------------------------------
TOTAL               289     23    92%
========================================================================================== 15 passed in 7.50s ==================================
```

8.3 End-to-End Integration Testing

End-to-end testing was conducted by exercising the full user flow from the React Native frontend on a physical Android device running Expo Go:

- User registration: A new user account was created successfully, with data persisted to the PostgreSQL database and viewable in the VS Code PostgreSQL extension.
- User login: Existing users could log in with their email and password. JWTs were correctly issued and stored in the frontend state.
- Schedule creation: Tablet schedules were created through the frontend and retrieved correctly, displaying the correct tablet names in the appropriate time-of-day sections.
- Tablet prediction: Images of the four tablet classes were selected from the device photo library and submitted to the predict endpoint. After updating the backend with the final trained model, all four tablets were correctly classified.
- Schedule comparison: Following correct tablet prediction, the frontend correctly compared the predicted tablet against the user's schedule and displayed the appropriate message (required or not required).
- Intake logging: The 'Mark as Taken' button successfully logged the intake to the database, and the logged entry was visible in today's history.

A limitation which I identified during end-to-end testing was that when tablet images sourced from the internet (rather than the Kaggle dataset) were used, some misclassifications occurred. Specifically, DayZinc was misclassified as Bioflu in one test case, and a Bioflu image was classified as Bactidol in another. These errors are attributed to the domain gap between the synthetic training images and real-world photographs. This is a known limitation of models trained on synthetic datasets, and I have discussed this issue further in the conclusion.

9 Ethics

The Tablet Classification App operates in the intersection of healthcare and technology, and accordingly raises a number of significant ethical considerations.

Data Privacy and Security. The application handles sensitive personal health data, specifically information about a user's medication schedule and intake history. User passwords are stored as bcrypt hashes and are never stored or transmitted in plain text. JWTs are used for session authentication, limiting the need to store session state on the server. However, the medication schedule and intake log data stored in PostgreSQL is not currently encrypted at rest. Before this application could be deployed to users in a real-world context, appropriate data protection measures would need to be implemented: encryption of sensitive fields at rest, secure database backups, HTTPS enforcement for all communications, and a data retention policy compliant with the General Data Protection Regulation (GDPR) and any applicable national health data regulations.

Accessibility for the Target User Group. The login mechanism presents a genuine ethical concern for the intended user group. A person with memory difficulties - the primary target user described in the introduction - may be unable to reliably recall a username and password. The current login requirement could exclude precisely the people the application is most intended to help. This issue was identified and acknowledged following the Christmas demonstration. The proposed future solution is biometric authentication using Face ID or fingerprint recognition, available through the expo-local-authentication Expo package.

Machine Learning Accuracy and Medical Risk. The most significant ethical concern is the accuracy of the machine learning model in a medical context. Incorrect tablet identification could, in a real-world deployment, lead a user to take the wrong medication or fail to take a required one. Both outcomes could have serious health consequences. The current model is a proof of concept trained on four synthetic image classes. It performs well on in-distribution data (images from the same Kaggle dataset) but shows some misclassification on out-of-distribution images (real-world photographs). Before any clinical or commercial deployment, the model would require extensive testing with a much larger and more diverse real-world dataset; independent validation by medical and regulatory experts; and review under applicable medical device regulatory frameworks (such as the EU Medical Device Regulation for software as a medical device, or equivalent frameworks in other jurisdictions). The application as currently developed is explicitly a proof-of-concept demonstration and is not intended for clinical use.

Environmental Impact. The training of deep learning models is computationally intensive and, by extension, consumes electrical energy. The environmental cost of large-scale machine learning training has been recognised as a growing concern in the field. For this project, training was conducted on Google Colab, which uses shared cloud infrastructure. While this is not carbon-neutral, it is substantially more efficient than training on dedicated on-premises GPU hardware that would otherwise be idle. The number of training runs was also kept to what was necessary for development progress, rather than exhaustive hyperparameter sweeps.

Data Source and Consent. The training data used for the machine learning model is a publicly available synthetic dataset from Kaggle. No personal data, patient data, or images of real patients were used in training. The synthetic nature of the dataset means there are no privacy concerns associated with the training data itself.

10 Future Work

Several areas have been identified for future development:

- Biometric login: Implementing Face ID or fingerprint authentication to remove the password dependency for users with memory difficulties. This was identified as the most important accessibility improvement and should be the first enhancement to implement.
- Push notification reminders: Adding scheduled push notifications to alert users when a tablet is due. The current application only helps a user who has already remembered to open the app; notifications would address the more fundamental problem of forgetting to take medication.
- Model improvement: Expanding the model to classify a larger number of real-world tablet types using a higher-quality real-world photographic dataset. The current synthetic-image dataset produces a model that generalises imperfectly to real photographs.
- Data protection and GDPR compliance: Implementing encryption at rest for sensitive database fields, establishing a data retention policy, and conducting a full GDPR compliance review before any public deployment.
- Expanded test coverage: Increasing the breadth of the automated test suite to cover more edge cases and error conditions across both backend and frontend.
- Caregiver access: Adding a secondary account role for caregivers who can monitor a patient's intake log remotely, extending the usefulness of the application for those who assist others in managing their medication.

The project achieved its primary objectives and provided a strong foundation for all of these future enhancements. The experience of building a complete, integrated system across machine learning, backend, database, and mobile frontend layers within a single academic year project has been challenging but highly rewarding.

11 Conclusion

This project successfully delivered a working Tablet Classification App comprising a trained deep learning classification model, a RESTful backend API with secure authentication, a PostgreSQL relational database, and a React Native mobile frontend. The core goal of enabling a user to photograph a tablet, receive an identification powered by a machine learning model, and be informed whether the tablet is required according to their personal schedule was achieved and demonstrated end-to-end on a physical device.

The project provided extensive learning across multiple areas of software to me. The machine learning component was by far the most challenging and time-consuming area of the project. The iterative process of training, evaluating, diagnosing failures, and adjusting the model was enjoyable as I got a clear view of what affects the model’s performance. Key learnings included the impact of batch size on generalisation, the importance of examining per-class metrics (precision, recall, F1) rather than relying solely on overall accuracy, the practical challenges of dataset quality and class similarity, and the substantial benefit of transfer learning with a pretrained backbone compared to training from scratch.

The backend development reinforced practical skills in API design, containerisation with Docker, database design, and secure authentication patterns. The choice of FastAPI proved to be well-suited to the project requirements, particularly its automatic documentation generation and native support for Pydantic validation. Working with Docker and PostgreSQL in a containerised environment provided valuable experience with the kind of deployment architecture used in professional software development.

The frontend development provided an introduction to React Native and Expo, and highlighted the challenges of mobile development including network configuration, cross-platform compatibility, and the differences between development and production environments.

12 References

[1] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[2] GeeksForGeeks, "Introduction to Convolutional Neural Networks," [Online]. Available: https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/.

[3] Amazon Web Services, "What is data labelling for machine learning?," [Online]. Available: https://aws.amazon.com/sagemaker/groundtruth/what-is-data-labeling/.

[4] V. Lanz09, "Pharmaceutical Drugs and Vitamins Synthetic Images," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/vencerlanz09/pharmaceutical-drugs-and-vitamins-synthetic-images.

[5] Roboflow, "What is ResNet-50?," [Online]. Available: https://blog.roboflow.com/what-is-resnet-50/.

[6] GeeksForGeeks, "Underfitting and Overfitting in Machine Learning," [Online]. Available: https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/.

[7] Hero Health, "Hero Smart Dispenser," [Online]. Available: https://herohealth.com/our-product/.

[8] Pivotell, "Pivotell Advance Dispenser," [Online]. Available: https://www.pivotell.co.uk/.

[9] TabTime, "Automatic Pill Dispenser," [Online]. Available: https://tabtime.com/products/automatic-pill-dispenser.

[10] GeeksForGeeks, "Comparing MySQL, PostgreSQL and MongoDB," [Online]. Available: https://www.geeksforgeeks.org/postgresql/comparing-mysql-postgresql-and-mongodb/.

[11] JWT.io, "JSON Web Tokens," [Online]. Available: https://www.jwt.io/.

[12] FullStack Labs, "Using React Native Camera in Your App," [Online]. Available: https://www.fullstack.com/labs/resources/blog/using-react-native-camera-in-your-app.

[13] AWS, "What is data augmentation?," [Online]. Available: https://aws.amazon.com/what-is/data-augmentation/.

[14] CodeSignal, "Generating JWT Tokens for Authentication," [Online]. Available: https://codesignal.com/learn/courses/mastering-fastapi-authentication-with-jwt/.

[15] IBM, "What is API Testing?," [Online]. Available: https://www.ibm.com/think/topics/api-testing.

[16] Mozilla Developer Network, "HTTP 422 Unprocessable Entity," [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/422.

Appendix A: Weekly Development Logs Summary

The following provides a summary of the weekly development logs maintained throughout the project in OneNote. Full logs including screenshots, code snippets, and architecture diagrams are available in my Project Engineering OneNote.

Week	Summary
Week 1 (Sept)	Project concept exploration. Two initial ideas considered: a suncream booth and an automated tablet dispenser. Decided to investigate hardware tablet dispenser further.
Week 2 (Sept)	Tested feasibility of hardware tablet dispenser. Three design iterations documented with sketches. Failed to settle on a final design. Concerns arose about how to ensure only one tablet is dispensed.
Week 3 (Oct)	Supervisor meeting: advised to be careful with hardware projects due to time cost. Pivoted to software-only ML approach. Research into existing products: TabTime, Pivotell, Hero Smart Dispenser, Smart Pill ID, Drugs.com, Pharmasee. Preferred the Hero Smart Dispenser design concept.
Week 4 (Oct)	Final project design chosen. Project proposal written and submitted to Moodle. Architecture diagram and timeline completed. Sprint 1 closed on Jira.
Week 5 (Oct)	Reflected on proposal. Informed that project deadline is Easter midterm, not end of semester 2. Sprint 2 planned on Jira. Plan adjusted to accommodate earlier final code deadline.
Week 6 (Nov)	Database comparison: MySQL vs PostgreSQL vs MongoDB. PostgreSQL selected due to full ACID compliance, data integrity, and recent experience from CICD module.
Week 7 (Nov)	React Native research. CNN research. Backend user login and authentication development started. Python FastAPI microservice established.
Week 8 (Nov)	CRUD endpoints designed: POST for user creation, GET for user retrieval (all and by ID), PUT for updates, DELETE for removal. PostgreSQL setup in VS Code.
Week 9 (Nov)	Sprint 2 delayed due to exam period. Limited project progress.
Week 10 (Nov)	User registration endpoint working. JWT research completed. PostgreSQL data visible in VS Code extension. Registration data saved to users table.
Week 11 (Dec)	JWT authentication completed. Bcrypt 72-byte password truncation issue resolved. Docker port conflict: three containers from previous sessions occupying ports. Resolved by deleting containers in Docker Desktop.
Week 12 (Dec)	Login endpoint tested and working for existing users. Validation error on Swagger (email vs username field) identified and resolved. Architecture diagram redesigned with larger font and technology labels for Christmas demo.
Christmas Break (Jan)	Christmas demo result: 40%. Critical flaw: app requires password login from memory-impaired users. Proposed Face ID as future feature. Backend endpoint development continued. JWT token generation working.
S2 Week 1 (Jan)	Machine learning development started. Found tablet image dataset on Kaggle. Imported to Google Colab. Split into training/validation sets. Tablet schedule endpoint development.
S2 Week 2 (Jan)	Supervisor meeting: deprioritise Face ID; focus on ML. Use a small dataset (10 tablets) to show high accuracy. Need to get ML working this week. Small

	custom dataset of 3 tablets photographed then abandoned in favour of Kaggle dataset.
S2 Week 3 (Feb)	CNN training on 10-class Kaggle dataset. 10 epochs. Accuracy/loss plots drawn with Matplotlib. Training accuracy improving but validation accuracy poor. Overfitting identified.
S2 Week 4 (Feb)	Epoch experimentation: 2, 10, 15 epochs tested. 15 epochs: 97.19% training, 58.67% validation. Shows clear overfitting despite high training accuracy.
S2 Week 5 (Feb)	Data split experimentation (80/10/10, 70/15/15, larger validation). 30 epochs: 97.75% training, 51.22% validation. This is worse than last week's results worse. 15 epochs with 70/15/15 split better. Overfitting confirmed.
S2 Week 6 (Feb)	Frontend work started (React Native, Expo). Batch size reduced 32→16: validation accuracy improved by ~19%. AdamW optimiser adopted. Transfer learning (ResNet50) research begun.
S2 Week 7 (Mar)	ResNet50 transfer learning adopted after F1 scores and confusion matrix showed persistent class confusion. Layer freezing error discovered: too many layers frozen. Error corrected by setting trainable=True for all layers.
S2 Week 8 (Mar)	4 classes selected based on F1 scores. preprocess_input applied to all batches. New confusion matrix significantly cleaner. F1 scores improved.
S2 Week 9 (Mar)	Visual inspection of all 10 tablet classes. 4 most visually distinct selected: Alaxan, Bactidol, Fish Oil, Bioflu (later revised to Bactidol, Bioflu, DayZinc, Medicol). Fish Oil naming issue (space in folder name) caused loading error. This issue was debugged.
S2 Week 10 (Mar)	1000-epoch target with early stopping (patience=5). Model stopped at 27 epochs. Confusion matrix much improved. Model exported to backend. Prediction endpoint tested on Swagger with dataset images. Android Studio updated. Frontend camera integration via expo-image-picker.
Easter Week 1 (Apr)	Frontend restructured in JavaScript (TypeScript template abandoned). Schedule stack navigator implemented. Image picker integrated. Prediction endpoint works end-to-end. Intake logging button added. Expo Go network issue diagnosed and resolved.
Easter Week 2 (Apr)	Final poster completed on Canva with supervisor feedback incorporated. API test suite (pytest) developed with monkeypatched ML model and in-memory SQLite test database. Report writing.
Week 12 S2 (Apr)	Report writing continued. All major project components working. Final testing completed.

Appendix B: Software and Technologies Used

Technology	Purpose
React Native / Expo	Mobile frontend framework and cross-platform development toolchain
JavaScript	Frontend programming language
Python FastAPI	Backend REST API framework
Uvicorn	ASGI server for running the FastAPI application
PostgreSQL	Relational database for user accounts, schedules and intake logs
SQLAlchemy	Python ORM for database interaction
Pydantic	Data validation and serialisation for API request/response models
Docker / docker-compose	Containerisation of backend and database services
Keras / TensorFlow	Deep learning model training and inference
ResNet50 (ImageNet pretrained)	Pretrained CNN backbone for transfer learning
Google Colab	Cloud-based GPU environment for ML model training
python-jose	JWT generation and verification
passlib / bcrypt	Secure password hashing
expo-image-picker	Camera and photo library access on mobile device
React Navigation	Screen navigation management in the React Native app
pytest	API test suite execution framework
Jira	Agile project management and sprint tracking
Visual Studio Code	Primary development environment
OneNote	Weekly development log maintenance
Canva	Project poster design
Swagger	Advanced API endpoint testing